1. You're designing a library system in C++, but this isn't just any boring old database—this is where books come to life (well, sort of). First, you need a Book class because, let's be honest, a library without books is just a fancy warehouse. This class will hold the VIP details: title, author, and publisher—the holy trinity of bookish existence. Of course, like any respectable book, it needs a constructor to bring it into the world and a display function to proudly announce, "Hey, look at me! I exist!" But hold onto your bookmarks, because here comes the real page-turner! You need a FictionBook class that inherits from Book—because let's face it, fiction books are the fun ones! This subclass adds genre (so we know if it's a thrilling mystery or a sappy romance) and protagonist (because every story needs a hero—or an anti-hero if you're feeling dramatic). This class will also have a constructor to set everything up and a display function to spill all the juicy details. Finally, in the main function, you'll create an object of FictionBook and display its details, because what's the point of writing code if you can't flex it? So, grab your imaginary reading glasses, and let's turn this coding project into a bestseller! 📖💻✨

2. You're about to craft the greatest game world ever (or at least one that compiles). It all starts with the legendary Character class—the grand ancestor of all fighters, with two crucial stats: health (so they don't collapse after a light breeze) and damage (because tickling enemies doesn't win battles). Of course, this class needs a constructor to give life to these brave souls and a display function so they can proudly show off their stats like RPG influencers. But what's a game without good guys and bad guys? Enter the Enemy and Player classes, both inheriting from Character. The Enemy exists solely to make players rage-quit, while the Player is the hero who either saves the world or just hoards loot for hours. Both get their own constructors and display functions—because villains and heroes alike deserve their moment in the spotlight. Now, hold onto your enchanted capes because here comes the real magic—The Wizard! This arcane warrior inherits from Player but also wields magic power and spells, because swinging a sword is way too basic. Naturally, it gets a constructor to infuse it with all the mystical goodness, plus a display function so it can flex its sorcery like a true magical influencer. Finally, in your main function, you'll summon a Wizard object and display its stats, because if you're writing all this code and not showing off the results, did the magic even happen? So, grab your coding wand, chant "std::cout," and let's conjure up some legendary C++ wizardry! ✨🔥🧙♂️

3. You're building a next-level game engine, and you need a system to keep track of all the important stuff—like where things are and how badly they're injured. To do this, you decide to create two almighty base classes:
   - Position – This class ensures that your game objects know where they are in 3D space because teleporting randomly isn't a great gameplay feature (unless you're making a glitch simulator). It will have three trusty coordinates: x, y, and z—so objects don't accidentally fall into the void.
   - Health – Because what's a game without damage? This class keeps track of an object's health, making sure your characters don't just shrug off dragon attacks like minor inconveniences.

   Now, things get even more interesting. You create the Character class, which inherits from both Position and Health (because why settle for just one inheritance when you can have both?). This class will represent the heroes and warriors of your game, complete with their own special abilities and actions. Finally, in your main function, you'll create a Character object, set its position and health, and then show it off—because what's the point of writing all this glorious code if you don't get to see your hero standing (or limping) in 3D space? So, fire up your keyboard, prepare for some

legendary multiple inheritance, and let's build a game world that doesn't instantly collapse into chaos! 🎮🚀🔥

4. You've been assigned the ultimate challenge: developing a school management program that actually keeps track of students and teachers (because, apparently, shouting names during roll call isn't efficient). You start with a Person class, because everyone in a school—students, teachers, and that one mysterious janitor—has a name and an age. But life gets more complicated! Students have a student ID (to misplace their library books officially) and a grade level (because calling everyone "kid" isn't ideal), while teachers have a subject (which they lecture about with varying enthusiasm) and a room number (where they trap students for extra assignments). Naturally, you create Student and Teacher classes that inherit from Person and add their own special attributes. But here's where things get really wild: you need a GraduateStudent class that inherits from both Student and Teacher, because some poor souls are both students AND teaching assistants—learning by day, grading papers by night, and questioning their life choices at all times. Finally, in your main function, you create a GraduateStudent object and display its details, because if they have to suffer through hybrid inheritance, the least you can do is let them see their own existence printed out. Now, grab your keyboard, embrace the chaos of multiple inheritance, and let's make this school system as smart as the overworked graduate students running it! 🎓📚💻

5. You've been given the noble task of creating a vehicle management program—because tracking cars in your head is so last century. You start with a Vehicle class, the granddaddy of all things on wheels, which proudly holds a make, model, and year, because even the rustiest old truck deserves an identity. Naturally, you add a constructor to initialize these values because, well, cars don't just appear out of thin air (unless it's a sci-fi movie). But wait—here comes Car, a fancier, more detailed version of Vehicle that adds the number of doors (so we know if it's a sleek two-door sports car or a minivan full of screaming toddlers) and fuel efficiency (because let's be honest, gas prices aren't getting any cheaper). This class, of course, calls the Vehicle constructor in style using an initialization list, because that's how classy programmers roll. But the real game-changer? The ElectricCar class! ⚡🚗 Inherits from Car, ditches gas, and brings in a shiny new variable: battery life—so you know exactly how far you can go before panicking about the next charging station. Like a true inheritor, ElectricCar also makes use of the initialization list, because efficiency isn't just for batteries. Finally, in the main function, you create an ElectricCar object and display its details—because after writing all this code, the least you deserve is seeing a virtual Tesla (or a budget electric scooter) come to life. Now, plug in your C++ skills, buckle up for some inheritance magic, and let's get this coding road trip started! 🚗💨💻

6. You've been hired by a fictional tech company that believes "more features = better product", and your job is to design the ultimate multi-purpose gadget. You start with a base class called Device, because every overpriced gadget—whether it's a smartphone, tablet, or toaster that plays Spotify—needs a brand and model. So far, so good. Then, the marketing team gets involved. They demand a Smartphone class that adds a SIM slot count (because dual SIM = instant genius) and a Tablet class that adds stylus support (so people can pretend they're productive). Both of these classes inherit from Device, because duh, they're devices. But hold on—things are about to go off the rails. The CEO, in a caffeine-fueled eureka moment, demands a HybridDevice—a device that's both a Smartphone and a Tablet. You sigh, create a HybridDevice class that inherits from both Smartphone and Tablet, and BAM—welcome to the Diamond Problem™! 🚨 Your HybridDevice

now has two separate copies of the Device class, and your compiler is looking at you like, "What have you done?" 😟💥 Now, your mission (if you choose to accept it): fix this disaster using virtual inheritance, so your HybridDevice doesn't cause a compiler meltdown. Finally, demonstrate how to create and use a HybridDevice object without the program catching fire.